ChatGPT

Executive Summary

The **webserv** project (at 42 school) tasks the student with implementing a custom **HTTP/1.1 web server in C/C++98**. The server must parse a user-specified configuration file, listen on one or more ports, and correctly handle at least the GET, POST and DELETE methods (including file upload and CGI execution). It must serve static files with proper MIME types and default error pages, and support persistent connections and chunked transfer encoding per HTTP/1.1 standards [1] [2]. 42's evaluation emphasizes **correctness and robustness**: no crashes or memory leaks (any non-functional server gets a 0), adherence to HTTP and config requirements, non-blocking I/O with a single `poll` / `select` / `epoll` loop (misusing blocking calls or extra processes triggers 0) [1]. Deliverables include the server executable, a Makefile, configuration and sample content files, and a README in English with prescribed sections (title, description, instructions, references, logins, AI-use disclosures) [3].

This report analyzes the project requirements, lists the core concepts (with priority), and provides concrete resources (official subject, relevant RFCs, tutorials, example servers) to learn those concepts. We outline a phased implementation roadmap with milestones and estimated durations, include a Gantt-style timeline, discuss common pitfalls and test cases, and give optimization/security tips. Finally, we recommend development tools (linters, debuggers, test frameworks) and present a pre-submission checklist.

Project Goals and Evaluation Criteria

According to the official subject, the goal is to **write a non-blocking HTTP/1.1 server in C++98** that reads a configuration file (modeled after Nginx) and hosts static and dynamic content. Key functional requirements (mandatory part) include:

- **Static file serving:** Serve files (HTML, images, etc.) from server-root directories specified in the config. Map URIs to filesystem paths and handle index files and directory listings if enabled.
- **HTTP methods:** At minimum, correctly implement GET (retrieve resource), POST (receive/upload resource), and DELETE (remove resource) [4] [5]. For example, a successful GET returns status 200 with the file contents; a DELETE should remove the file and respond with 200/204 [4] [5].
- **Persistent connections:** Support HTTP/1.1 persistent (keep-alive) connections by default [1]. This allows multiple requests per TCP connection; only close when a "Connection: close" header is seen or on protocol version downgrade [1].
- **Chunked transfer decoding:** Accept and decode chunked request bodies (RFC 7230, §4.1). The server must read the chunk sizes and assemble the full request payload before passing it to handlers (e.g. CGI) [2].
- **CGI execution:** Support at least one CGI language (e.g. PHP or Python). On a matching extension or directory (like `/cgi-bin/`), fork/exec a child process with the correct environment variables and pass the request body to its stdin. Return the CGI's stdout as the HTTP response. Note: CGI scripts should see request data and env (e.g. `CONTENT_LENGTH`) [2] [6].

- **Configuration file:** Parse a text config file defining one or more virtual servers. Directives include `listen` ports, `root` directory, allowed methods, redirections, index page, upload path, etc. The server must handle multiple `listen` ports (via the config) simultaneously.
- **Concurrency model:** Use **non-blocking I/O** and a single I/O multiplexing loop (e.g. `poll` / `select` on Linux or `kqueue` on macOS). Do not use one thread/process per connection. All sockets (listen and clients) must be non-blocking, and **all readiness must be handled via the poll loop** [1]. Any blocking read/write or spawning additional threads/processes for clients will fail the evaluation.
- **Robustness:** The server must not crash or leak memory. Incorrect behavior (unhandled requests, wrong headers) or resource leaks result in major penalties [1]. Score 0 if the server dies under valid test.
- **Performance:** The server should remain available under load (stress-tested with tools like siege). A single `poll` loop should efficiently handle many concurrent clients.
- **Documentation:** A complete README.md in English with prescribed sections (project description, build/run instructions, references including any AI usage, author logins in italics on first line) is required [3].

Assumptions: The subject specifies C++98; we assume using standard library (`<iostream>`, `<string>`, etc.) and POSIX system calls. No external HTTP libraries (must hand-roll parsing). We assume operation on Linux/macOS. If the official rubric is not public, we assume the above list covers the primary grading points.

Core Concepts and Technologies

Key topics to master for this project include:

- **Sockets and TCP/IP (High):** Understanding BSD sockets (e.g. `socket()`, `bind()`, `listen()`, `accept()`, `connect()`) is essential for network I/O. You must know how to create a listening socket and accept client connections [7] [8]. Learning resources: Beej's Guide (network programming basics) [9]. *(Priority: High)*

- **HTTP/1.1 Protocol (High):** Core of the project. HTTP is an application-layer protocol with a request/response format [10] [1]. You need to parse request lines (method, URI, version) and headers, and generate valid responses (status line, headers, body). Read [RFC 7230] (message syntax, persistent connections, header fields) and [RFC 7231] (methods and status codes) for authoritative details. HTTP is stateless by default [10]. *(Priority: High)*

- **HTTP Methods & Status Codes (High):** Specifically, implement **GET** (retrieve resource), **POST** (submit data, e.g. file upload), and **DELETE** (remove resource) per [RFC7231] [4] [5]. Understand safe/idempotent methods (GET/DELETE are idempotent; POST is not). Know key status codes: 200 OK, 201 Created, 204 No Content, 301/302 (redirect), 400 Bad Request, 404 Not Found, 405 Method Not Allowed, 413 Request Entity Too Large, 500 Internal Server Error, etc. [RFC7231] defines status-code semantics and classes [11] [12]. *(Priority: High)*

- **HTTP Headers and MIME types (High):** Properly handle common headers: `Content-Length`, `Transfer-Encoding`, `Connection`, `Host`, `Content-Type`, etc. Use MIME types (e.g. from `/etc/mime.types`) to set the `Content-Type` for static files. The Python

`SimpleHTTPRequestHandler` example shows how MIME types map file extensions [13]. *(Priority: High)*

- **Chunked Transfer Encoding (High):** HTTP/1.1 can send request or response bodies in "chunked" encoding [2]. You must implement decoding of chunked request bodies (removing chunk-size headers and assembling full payload). RFC 7230 §4.1 explains that chunked encoding wraps the payload in length-prefixed chunks [2]. Practice by reading [RFC7230] (section 4.1) [2].

- **Persistent Connections (High):** By default, HTTP/1.1 uses **persistent (keep-alive) connections**, meaning multiple requests/responses over one TCP connection [1]. Implement this: after sending a response, keep the socket open until the client sends `Connection: close` or the protocol version changes. [RFC7230] §6.3 notes that HTTP/1.1 defaults to persistent connections [1]. Failing to fully drain request bodies or incorrectly closing a socket will break persistence. *(Priority: High)*

- **Request Parsing & Response Generation (High):** Write code to **parse** the HTTP request text (request line + headers + optional body) and **generate** a correct HTTP response string. Pay attention to CRLF vs LF, case-insensitive header names, handling of request bodies (based on `Content-Length` or `chunked`). Response generation must include status line, mandatory headers, an empty line, then body. For example, a simple GET response includes `HTTP/1.1 200 OK`, `Content-Type`, `Content-Length`, blank line, then file bytes.

- **File I/O and Directories (High):** Serving static files requires using file I/O syscalls (e.g. `open/read/close` or C++ `std::ifstream`) to read files from disk. Handle directory requests by looking for an index file (e.g. `index.html`). If directory listing is enabled, generate an HTML list of files; otherwise return 403 Forbidden or a default page. For POST file upload, read the request body and write it to a file. *(Priority: High)*

- **CGI Execution (High):** For dynamic content, use the Common Gateway Interface (CGI). This involves `fork()`-ing a child process and `exec()`-ing the interpreter (e.g. `/usr/bin/python` or `php-cgi`) with environment variables (REQUEST_METHOD, QUERY_STRING, CONTENT_LENGTH, etc.) set per [RFC3875 – CGI/1.1] or examples. Python's `CGIHTTPRequestHandler` demonstrates the idea: it runs scripts in `/cgi-bin/` and captures their output [6]. Your server must feed the request body to the CGI's stdin and send the CGI's stdout back to the client. *(Priority: High)*

- **Concurrency / Event Loop (High):** The server must be **single-process, non-blocking**. Use an event-driven model: register sockets with `select()`/`poll()` or, preferably on Linux, `epoll()`. Monitor all listening sockets and connected clients for readiness to read/write. Handle each ready event without blocking (use non-blocking sockets). Do *not* spawn one thread/process per client. On macOS, use `kqueue` as the equivalent. Conceptually, this is the Reactor pattern. *(Priority: High)*

- `select`**/**`poll`**/**`epoll` **vs Threads/Processes (High):** Understand the trade-offs: a multiplexed I/O model (select/poll/epoll) can scale to many connections without threading overhead, whereas one-thread-per-connection or one-process-per-connection is simpler but resource-heavy. This project *requires* the event-loop approach. We compare models below in a table. *(Priority: High)*

- **Configuration File Parsing (High):** Write a parser for the custom config syntax (similar to Nginx). Understand tokens, braces, and directives. For example, allow blocks like: `server { listen 8080; root /var/www; index index.html; ... }`. You may use string/token parsing or stream input. Ensure robust error checking (invalid configs → error). *(Priority: High)*

- **Concurrency Models (Medium):** While the project mandates an event loop, understand other models for context. Compare **process-per-connection (fork)** and **thread-per-connection** to the event-driven model. Table provided below. *(Priority: Medium)*

- **Memory Management (Medium):** C++98 and manual memory management means you must avoid leaks (free `new`/`malloc`, close FDs). Use RAII (smart pointers, `std::vector` or `std::string`) to manage resources. Tools like Valgrind should be used. Memory leaks or buffer overruns are critical faults. *(Priority: Medium)*

- **Security Basics (Medium):** At minimum, prevent **path traversal** (reject URLs with `..` that escape root). Avoid command injection in CGI (sanitize inputs). Use safe functions (e.g. `snprintf` over `strcpy`). If running as root to bind privileged ports, drop privileges if possible. While full security hardening is beyond scope, be aware. *(Priority: Medium)*

- **Development Environment (Low):** Learn to use `g++ -std=c++98 -Wall -Wextra -Werror` to compile, and a `Makefile` to automate builds. Familiarize with 42's coding style if required (norminette is for C, not C++). *(Priority: Low)*

Learning Resources

Official Materials: The primary specification is the **42 school webserv subject** (usually on the 42 intranet). A publicly available mirror (or summary) can be found in the [cclaude42/webserv GitHub repo] [3], which includes the project description and links. Read the subject carefully.
RFCs: The authoritative HTTP/1.1 standards are RFC 7230–7235. Key sections to study:

- *RFC 7230* (HTTP/1.1 Message Syntax and Routing – start line, headers, chunked encoding, persistence) [2] [1].
- *RFC 7231* (HTTP/1.1 Semantics and Content – methods, status codes) [4] [11].
- *RFC 7232* (Conditional Requests – for understanding caching headers).
- *RFC 7233* (Range Requests – how to handle `Range:` if implementing partial GET; optional).
- *RFC 7234* (Caching).
- *RFC 7235* (Authentication – for HTTP auth headers if considered; mostly bonus).
  Links: [RFC7230][20], [RFC7231][33], [RFC7232][49] (and similarly [RFC7233], [RFC7234], [RFC7235] via rfc-editor).

Tutorials and Guides:

- **Beej's Guide to Network Programming** (C/C++ socket tutorial) [9]. Covers `socket()`, `bind()`, `listen()`, `accept()`, and non-blocking I/O.
- **Ncona's "Building a Simple Server with C++"** [7] [8] – a step-by-step blog showing socket setup and accepting a connection.

- **MDN Web Docs – HTTP Overview** (Mozilla Developer Network) – high-level introduction to HTTP concepts. Useful quick reference on requests and responses.
- **Tutorialspoint or GeeksForGeeks HTTP tutorials** – basic coverage of HTTP protocol and headers.
- **C++ HTTP Server Examples:**
- *trungams/http-server* (GitHub) [14] – a simple C++ HTTP/1.1 server project (uses multithreading, but shows HTTP message handling).
- *bloominstituteoftechnology/C-Web-Server* [15] – a teaching example in C (with parser and cache) illustrating request parsing and response generation.
- *Python* `http.server` *docs* [16] [6] – explains Python's built-in `SimpleHTTPRequestHandler` (static files) and `CGIHTTPRequestHandler` (CGI scripts). Good for understanding request flow and CGI.

Books: If available, consult *"Unix Network Programming"* (Stevens) or *"HTTP: The Definitive Guide"* for in-depth theory. For quick web references, Wikipedia's [HTTP page][47] or Wikipedia on CGI and MIME types can provide overviews.

Implementation Roadmap and Timeline

A phased approach is recommended. Below is a suggested ordering of milestones, with estimated duration for a part-time student. Adjust as needed. Each milestone has clear deliverables.

1. **Foundations (1–2 weeks):**

2. *Learn basics:* Review HTTP 1.1 protocol and BSD sockets (per resources above). Set up a development environment (compiler, debugger, Valgrind). Draft a high-level design (data structures for requests, server class outline).

3. *Deliverable:* A written design doc outline; environment ready.

4. **Basic Server Skeleton (2 weeks):**

5. Implement a simple TCP server: open a socket, `bind()` and `listen()`, then `accept()` connections. In the handler, read raw data and respond with a hardcoded HTTP response (e.g. "HTTP/1.1 200 OK\r\nContent-Length: 5\r\n\r\nHello").

6. **Test:** Use `curl` or `telnet` to verify the server responds.

7. *Deliverable:* C++ program `webserv` that listens on a port and serves a fixed message.

8. **HTTP Request Parsing and Static GET (2 weeks):**

9. Extend the handler to parse the HTTP request line and headers. Extract method, URI, and HTTP version. For GET: convert the URI to a local file path (join with document root, handle `/` and `../`). If file exists and is readable, read its contents and return a `200 OK` with correct `Content-Type` and `Content-Length`. If not found, return `404 Not Found`. If directory: optionally look for

`index.html`; if none, return 403 or auto-generate directory listing. Also implement `HEAD` by returning headers without body.

10. *Deliverable:* Serving static files based on config directory. No dynamic or config parsing yet.

11. **Configuration Parsing and Multi-Server (1–2 weeks):**

12. Define a syntax for the config file (per subject requirements). Write code to parse it: supporting blocks (e.g. `server { ... }`), directives like `listen <port>`, `root <path>`, `index <file>`, `methods GET POST DELETE`, `cgi <dir>` or `fastcgi`, etc. Store the parsed config in C++ structs.

13. Update the server to use the config: open a listening socket for each `listen` port, and serve content according to each server block's settings (e.g. `root`). Handle at least two simultaneous ports in one process.

14. *Deliverable:* Config file support; server binds to configured ports and serves multiple "virtual hosts."

15. **POST, DELETE, and File Upload (1 week):**

16. Implement **POST**: If `Content-Length` is present and method is POST to a designated upload path or CGI, read the request body and either save it as a file or pass to CGI. Return 201 Created or 200 OK.

17. Implement **DELETE**: If DELETE is called on a resource, attempt to remove the file (`unlink()`), and return 204 No Content if successful or 404 if not found.

18. *Deliverable:* Functional POST and DELETE with basic payload handling.

19. **CGI Support (2 weeks):**

20. Identify when a request targets a CGI resource (by extension or URL path). Implement CGI by forking a child process with `fork()`: set up environment (e.g. `environ`) per CGI spec, write request body to child's stdin, and capture stdout. Non-blocking caution: you may need to temporarily pause non-blocking on the CGI pipe. Include correct `Status:` header if provided by CGI output.

21. *Test:* Use a simple Python or PHP script (e.g. outputting HTML or environment variables) to verify.

22. *Deliverable:* CGI scripts execute and their output is sent as the HTTP response.

23. **Concurrency and Non-Blocking I/O (2 weeks):**

24. Refactor the server into an **event loop**. Make all sockets non-blocking (use `fcntl`). Use `poll()` (or `epoll` on Linux) to wait on all listening and client sockets. Handle events: accept new connections, read incoming requests (handle partial reads!), write responses (handle partial writes). Ensure you never call a blocking I/O function on a socket.

25. Handle disconnects and clean up file descriptors.

26. *Deliverable:* A fully asynchronous server that serves all supported features without blocking.

27. **Testing, Optimization, and Hardening (1–2 weeks):**

28. Write unit or integration tests (see next section) and use tools: run under `valgrind` / AddressSanitizer to fix leaks, use `gdb` to debug crashes. Benchmark with `siege` / `wrk` for concurrency.

29. Optimize: consider using `sendfile()` for file transfer to speed up static files (zero-copy). Implement caching of open file descriptors or compiled CGIs if needed. Add logging if desired.
30. Ensure error handling: catch exceptions, handle bad requests (return 400), large headers or bodies gracefully (413 Payload Too Large).

31. *Deliverable:* Tested, optimized server; all memory errors fixed; performance tuned.

32. **Documentation and Submission (ongoing):**

33. Maintain a **README.md**: include project description (one paragraph summarizing goals) and clearly list build and run instructions (how to invoke `./webserv [config]`). Add references (RFCs, tutorials used) and note if any AI (LLM) assistance was used (per 42 policy).

34. Format and style: comply with any coding standards specified. Write clean Makefile.
35. Final deliverable: All source code, Makefile, sample configs and site files, and README in one repository.

The following Gantt chart outlines this schedule (assuming an early-May start and a part-time pace). Note that some tasks can overlap (e.g. writing README early):

```
gantt
    title Webserv Project Roadmap
    dateFormat  YYYY-MM-DD
    section Foundations
    Study HTTP/Sockets       :a1, 2026-05-01, 2w
    Draft Design             :a2, after a1, 1w
    section Development
    Basic TCP Server         :b1, after a2, 2w
    HTTP Parsing & Static    :b2, after b1, 2w
    Config File Parsing      :b3, after b2, 2w
    POST/DELETE Support      :b4, after b3, 1w
    CGI Support              :b5, after b4, 2w
    Event Loop (poll/epoll)  :b6, after b5, 2w
    Testing & Optimization   :b7, after b6, 1w
    section Delivery
    Documentation & Polish   :c1, after b7, 1w
```

Milestones: At each phase complete a demonstrable feature set as listed. Adjust timelines to your pace; this is a guideline.

Common Pitfalls, Testing, and Optimization

Common Pitfalls: Be aware of frequent errors:

- **Blocking I/O:** Accidentally using blocking reads/writes on a non-ready socket will stall or crash the server. Always check readiness via `poll()` before reading or writing. Grades of 0 are given for blocking socket calls [1].
- **Partial Reads/Writes:** TCP reads may return less data than requested. Loop until the full header or body is read (especially important for `Content-Length` bodies). Similarly, writes (`send()`) may send fewer bytes – loop until all bytes are sent.
- **CRLF and Header Parsing:** HTTP lines end in `\r\n`. Reading line-by-line carefully (stop at `\r\n\r\n`) is crucial. Mishandling CRLF vs LF can break parsing.
- **Memory Leaks:** Forgetting to `free` or close sockets/file handles will accumulate memory/FD leaks. Run under Valgrind or AddressSanitizer often.
- **Off-by-One / Buffer Overflows:** Avoid C-string overflows. Use safe alternatives or C++ `std::string`.
- **Path Traversal:** A request like `/../etc/passwd` must be sanitized and rejected, or normalize paths with `realpath()` and check root.
- **HTTP Strictness:** Some servers accept non-standard requests; 42 expects correct HTTP/1.1 behavior. For example, missing `Host` header or wrong CRLF sequences should be handled per spec (typically return 400).
- **Signal/Child Handling:** If using `fork()` for CGI, ensure you call `waitpid()` or handle `SIGCHLD` so zombies don't accumulate.

Test Cases (Validation): To verify correctness, use automated and manual tests:

- **Basic HTTP Tests:** Use `curl` or `wget` for simple scenarios: GET a static file (expect 200 and correct content/MIME), GET a non-existent file (404), HEAD request (headers only), GET a directory (index or 403).
- **Method Handling:** Test POST by uploading a file (e.g. `curl -d @file http://host/upload`), check that file is saved. Test DELETE on an existing and non-existing resource, and verify proper status (204 or 404).
- **Persistent Connections:** Use `curl -v --keepalive` and ensure multiple requests on one connection succeed. Verify that the server closes the socket when `Connection: close` is used.
- **Chunked Requests:** Emulate chunked transfer: e.g. `curl -X POST -H "Transfer-Encoding: chunked" --data-binary @file http://host/path`. Confirm the server correctly reads the data.
- **CGI:** Place a simple CGI script (e.g. a Python script that prints environment and some HTML). Access it via browser or `curl`. Check that QUERY_STRING and POST data are delivered to the script.
- **Concurrency/Stability:** Use a load tool (e.g. `siege` or `wrk`) with many parallel clients to GET or POST. The server must stay running and respond within a reasonable time.

- **Configuration:** Test the config file's directives: define two server blocks on different ports; ensure requests on each port obey that block's root and settings. Test invalid configs (parser should error out cleanly).
- **Edge Cases:** Send malformed requests to see if server returns 400 or handles them gracefully. Test very large headers/body (if supported, or return 413). Verify that graceful error pages are returned for 404, 500, etc.

Optimization and Security Tips:

- **Event Loop:** Use `epoll` (Linux) or `kqueue` (macOS) instead of `poll` for better scalability (especially with thousands of clients). Ensure proper `epoll_ctl` use (edge- vs level-triggered).
- **sendfile():** On Linux, `sendfile()` can send file contents directly over the socket without user-space copying. This speeds up static file serving.
- **Caching:** For frequently requested files or compiled scripts, consider caching results or file descriptors (though optional). For example, cache MIME type lookups or opened file streams.
- **Thread Safety:** If any global state is used, guard with locks or avoid sharing state. The recommended model is single-threaded, but if you ever use threads (e.g. for CGI), beware of data races.
- **Resource Limits:** Remember file descriptor limits (`ulimit -n`). 42 may test with many clients; you can increase this limit or use `epoll` to reduce per-connection overhead.
- **Security:** Always validate and sanitize inputs. Limit maximum request size to avoid DoS (could reject extremely large Content-Length). Do not execute CGI on arbitrary files – confine to `/cgi-bin/` or known extensions. Drop privileges (e.g. use `setuid`) if you bind low ports.
- **Code Quality:** Run static analyzers (`cppcheck`) and linters (`clang-tidy`) to catch potential bugs. Compile with warnings as errors.

Development Tools and CI

- **Compiler/IDE:** g++ (with `-std=c++98 -O2 -Wall -Wextra -Werror`). On macOS, use `clang++ -std=c++98`. Use an IDE or editor you're comfortable with (VSCode, CLion, vim, etc.).
- **Build Tools:** A Makefile is required. Use `make` to compile all `.cpp` files into `webserv`. Optionally use `cmake` for complex setups, but a plain Makefile is simplest.
- **Linters:** `cppcheck` or `clang-tidy` for C++ code issues. If following 42 "norm," ensure code style guidelines (though most 42 projects enforce norm for C only).
- **Debuggers:** `gdb` (Linux) or `lldb` (macOS) for stepping through. Check dynamic memory and I/O state on crashes.
- **Memory Checkers:** Run `valgrind --leak-check=full` to find leaks. Alternatively, compile with AddressSanitizer (`-fsanitize=address`) to catch out-of-bounds.
- **Network Test Tools:** Use `curl` or `httpie` for manual HTTP testing. Use `telnet` or `nc` for raw socket experiments.
- **Load Testing:** `siege`, `wrk`, or `ab` (Apache Bench) can simulate many clients.
- **Unit Testing:** For C++ components (parsing, etc.), consider GoogleTest or Catch2 to write unit tests. Not strictly required but helps catch bugs.
- **Continuous Integration (CI):** If submitting via Git/GitHub, set up a CI pipeline (GitHub Actions, Travis CI) to automatically build and run tests on each commit. Include steps to run valgrind or sanitizer checks and any regression tests.

- **Version Control:** Use `git` with meaningful commits. The final submission should have a clean history. Tag a version (e.g. `v1.0`) before submission.
- **Monitoring:** Logging can be invaluable. Consider adding verbose logs (to stderr) during development (connections, errors). Remove or minimize logs for final to avoid noise.

Concurrency Models Comparison

Model	Resource/ Memory Use	Scalability/ Performance	Complexity/Notes
Process-per-Connection (fork)	High (each client is a separate process with its own memory)	Poor at high scale (heavy context-switching, copying memory)	Simple to implement, strong isolation; however, not allowed by subject (except for CGI). Crashes in one child do not affect others, but expensive.
Thread-per-Connection	Moderate (threads share memory, but stack per thread)	Can handle more than forks, but still context-switch heavy	Easier than event loop; still requires locking for shared state. Not required here; threads would violate “single poll” rule.
Select/Poll (event-driven)	Low (single process, single thread; memory ~ data structures for fds)	Decent for hundreds of FDs; O(n) scan overhead	Handles many clients in one loop; simple conceptually but must manage state machines (partial reads). `select` is limited by max FD; `poll` removes that limit but still linear. Suitable for this project.
Epoll/Kqueue (event-driven)	Low (like select/ poll)	High (O(1) readiness notification, scalable to thousands of FDs)	Best performance under Linux (epoll) or BSD/Mac (kqueue). Involves registering fd events. Required knowledge of edge vs level triggering. Highly recommended for large-concurrency use.

Pre-Submission Checklist

Before submitting, verify the following:

- **Functionality:** All required features from the subject are implemented and work correctly (static files, GET/POST/DELETE, CGI, config settings).
- **Robustness:** The server does **not crash** on valid or unexpected input. It gracefully handles client disconnects and invalid requests (respond with appropriate 4xx/5xx codes).
- **Non-Blocking I/O:** All sockets are set to non-blocking and managed via a single `poll` / `select` / `epoll` loop. No accidental blocking calls.
- **Memory Safety:** No memory leaks or invalid memory accesses. Verified with Valgrind/ASan. All file descriptors (sockets, files) are closed appropriately.

- **HTTP Compliance:** Responses include correct status lines, headers (`Content-Length`, `Content-Type`, etc.) and use CRLF line endings. Chunked and keep-alive behavior follow RFC 7230.
- **Configuration File:** The server reads the config file (or default), and honors directives (ports, root, index, upload path, error pages, allowed methods, CGI dir). Multiple `listen` ports should work.
- **Error Pages:** Default or custom error pages for 400, 404, 500, etc. are served as required.
- **Makefile:** Compiles the code with `make` on 42's machines. It should use `-std=c++98`, `-Wall -Wextra -Werror`, and no forbidden libraries. No warnings or errors.
- **README:** Contains all required sections (title with logins, description, instructions, resources). First line with author logins in *italic* format (per 42 style).
- **References:** Document all sources and libraries used in README. If any AI (LLM) was used to generate code/analysis, explicitly cite it as required by 42 policy.
- **Testing:** Try submitting to a fresh environment (like the 42 VM) and test a few basic `curl` commands. Include sample configs and data to demonstrate features.
- **Code Style:** Follow C++ best practices for readability. Indentation, naming, and comments (where needed) should be clean.
- **Version Control:** All code should be in Git with a remote repository. No extra files (binaries, IDE settings). The final commit should have a clear message (e.g. "Final version").

If all items above are satisfied, the project is ready for submission.

Sources: The information above is drawn from the 42 webserv subject and standard HTTP/CGI references. Key sources include the [42 webserv GitHub subject summary] [3] and the HTTP/1.1 RFCs (e.g. RFC 7230 [2], RFC 7231 [4] [11]), as well as official docs like Python's `http.server` guide [6]. These have been used to ensure accuracy and completeness.

---

[1] [2] RFC 7230: Hypertext Transfer Protocol (HTTP/1.1): Message Syntax and Routing
https://www.rfc-editor.org/rfc/rfc7230.html

[3] GitHub - cclaude42/webserv: 42 - Coding a simple HTTP server in C++ · GitHub
https://github.com/cclaude42/webserv

[4] [5] [11] [12] RFC 7231: Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content
https://www.rfc-editor.org/rfc/rfc7231.html

[6] [13] [16] http.server — HTTP servers — Python 3.14.4 documentation
https://docs.python.org/3/library/http.server.html

[7] [8] Building a simple server with C++ – ncona.com – Learning about computers
https://ncona.com/2019/04/building-a-simple-server-with-cpp/

[9] Beej's Guide to Network Programming
https://beej.us/guide/bgnet/

[10] HTTP: Hypertext Transfer Protocol | MDN
https://developer.mozilla.org/en-US/docs/Web/HTTP

[14] GitHub - trungams/http-server: A simple HTTP/1.1 server implementation in C++ · GitHub
https://github.com/trungams/http-server

[15] GitHub - bloominstituteoftechnology/C-Web-Server: A simple webserver written in C · GitHub
https://github.com/bloominstituteoftechnology/C-Web-Server